

Sistema de Gestión Académica — Árbol Binario de Búsqueda (BST)

Universidad Técnica de Ambato

Asignatura: Estructura de Datos

Tema: Árboles Binarios de Búsqueda

Lenguaje: C++

Autor: Manobanda Masabanda Luis Alfredo

Curso: 3ro B

Carrera: Ingeniería en Software

Docente: Ing. José Caiza, Mg.



Introducción

Los árboles binarios de búsqueda (BST) son estructuras de datos fundamentales en la informática que permiten organizar, insertar y recuperar información de manera eficiente con una complejidad promedio de $O(\log n)$.

Este proyecto aplica dicha estructura para gestionar registros académicos de estudiantes de la Universidad Técnica de Ambato, utilizando la cédula de identidad como clave de ordenamiento. A través de su implementación en C++, se refuerzan conceptos esenciales como la recursividad, el manejo dinámico de memoria y los recorridos en árboles.



Objetivo General

Desarrollar un sistema de gestión académica en C++ basado en un Árbol Binario de Búsqueda que permita registrar, buscar, eliminar y analizar información de estudiantes de forma eficiente, aplicando los principios de las estructuras de datos dinámicas estudiados en la asignatura.



Objetivos Específicos

1. Implementar las operaciones fundamentales del BST

Implementar inserción, búsqueda y eliminación utilizando recursividad y manejo dinámico de punteros en C++, garantizando la integridad del árbol en cada operación.

2. Aplicar los recorridos del árbol

Utilizar los recorridos Inorden, Preorden, Postorden y BFS para visualizar la información de los estudiantes en diferentes estructuras de recorrido.

3. Incorporar funciones de análisis académico

Agregar funciones como conteo de nodos, cálculo de altura, búsqueda de notas máximas y mínimas, y filtrado de aprobados y reprobados.



Descripción del Proyecto

Sistema académico desarrollado en C++ utilizando un **Árbol Binario de Búsqueda (BST)** para gestionar estudiantes de la Universidad Técnica de Ambato.

Cada nodo almacena la información completa de un estudiante y la clave principal del árbol es la **cédula de identidad**.

El sistema permite:

- Insertar estudiantes
- Buscar estudiantes
- Eliminar registros
- Recorrer el árbol
- Analizar datos académicos



Conceptos Aplicados

Concepto	Descripción
BST	Árbol Binario de Búsqueda
Recursividad	Inserción, búsqueda, eliminación y recorridos
Punteros	Manejo dinámico de memoria
Queue	Utilizada para BFS
Struct	Nodo y Estudiante
Clases	Clase <code>ArbolBST</code>
Validaciones	Cédula, nota y nivel



Estructura del Repositorio

```
prueba-practica-arboles-cpp-java/  
├── src/  
│   └── sistema_estudiantes.cpp  
└──
```

```
├── capturas/
│   ├── captura_menu.png
│   ├── captura_insertar.png
│   ├── captura_recorridos.png
│   ├── captura_bfs.png
│   └── captura_funciones.png
└── README.md
```

Compilación y Ejecución

◆ Opción 1: Dev-C++

1. Abrir Dev-C++
2. Crear nuevo proyecto C++
3. Copiar el código fuente
4. Presionar **F9**

◆ Opción 2: Terminal con g++

Compilar

```
g++ -o sistema_estudiantes src/sistema_estudiantes.cpp
```

Ejecutar Linux / Mac

```
./sistema_estudiantes
```

Ejecutar Windows

```
sistema_estudiantes.exe
```

◆ Opción 3: Compilar con C++11

```
g++ -std=c++11 -o sistema_estudiantes src/sistema_estudiantes.cpp
```



Datos del Estudiante

Campo	Tipo	Descripción
cedula	string	Clave del BST
apellidos	string	Apellidos
nombres	string	Nombres
notaFinal	double	Nota final
carrera	string	Carrera
nivel	int	Nivel académico



Menú del Sistema

UNIVERSIDAD TÉCNICA DE AMBATO
Sistema de Gestión Académica - BST

1. Insertar estudiante
2. Buscar estudiante por cédula
3. Eliminar estudiante
4. Recorrido Inorden
5. Recorrido Preorden
6. Recorrido Postorden
7. Recorrido por niveles (BFS)
8. Contar estudiantes
9. Calcular altura del árbol
10. Mostrar estudiante con mayor nota
11. Mostrar estudiante con menor nota
12. Mostrar estudiantes aprobados
13. Mostrar estudiantes reprobados
0. Cargar datos de prueba
14. Salir



Recorridos Implementados

◆ Inorden

Izquierda → Raíz → Derecha

Muestra estudiantes ordenados por cédula.

◆ Preorden

Raíz → Izquierda → Derecha

Útil para copiar estructuras del árbol.

◆ Postorden

Izquierda → Derecha → Raíz

Utilizado para liberar memoria.

◆ BFS — Por niveles

Recorre el árbol nivel por nivel utilizando `queue<Nodo*>`.



Funciones Avanzadas

Función	Descripción
contarNodos()	Cuenta estudiantes
calcularAltura()	Calcula altura del árbol
buscarNotaMayor()	Busca nota máxima
buscarNotaMenor()	Busca nota mínima
mostrarAprobados()	Nota ≥ 7
mostrarReprobados()	Nota < 7



Criterios de Aprobación

- Nota mínima: **7.0 / 10**
 - El sistema muestra:
 - APROBADO ✓
 - REPROBADO ✗
-



Notas Técnicas

- Uso de memoria dinámica con **new** y **delete**
- Destructor para liberar memoria
- Eliminación mediante sucesor Inorden
- Validación de cédula y notas
- Funciones recursivas en todo el BST



Capturas de Pantalla

Captura	Descripción
captura_menu.png	Menú principal
captura_insertar.png	Inserción
captura_recorridos.png	Recorridos
captura_bfs.png	BFS
captura_funciones.png	Funciones avanzadas



Conclusiones

1.

El BST permitió gestionar estudiantes de manera eficiente mediante operaciones rápidas de inserción y búsqueda.

2.

La implementación en C++ fortaleció el manejo de punteros, memoria dinámica y recursividad.

3.

Los recorridos demostraron diferentes formas de visualizar la estructura del árbol según la necesidad del sistema.



Recomendaciones

1.

Implementar árboles AVL o Rojo-Negro para evitar desbalanceo.

2.

Agregar persistencia mediante archivos para conservar información.

3.

Mejorar validaciones y manejo de errores del sistema.



Autor

Luis Alfredo Manobanda Masabanda

Ingeniería en Software — UTA



Asignatura

Estructura de Datos — Universidad Técnica de Ambato — 2026